



# BASELINE SMMYE

## V1

Software Configuration Management

### Descripción breve

This document explains what the SMMyE project is, and its functional and non-functional requirements.

IGNACIO ANDRADE SALAZAR | 215822856  
Maestría en Ingeniería de Software.

## About project

This project consists of a web platform, called SMMyE (Sistema de Monitoreo Medioambiental y Edafológico), designed to acquire, store, visualize and interpret environmental and soil data collected by IoT nodes distributed across the Región Valles, Jalisco. The main objective of the system is to give researchers, farmers, local authorities and the general public an accessible way to understand the environmental and soil conditions of their region, using data captured continuously by sensor stations installed in the field.

SMMyE integrates a conversational assistant called EcoFlow, which combines a locally-hosted Large Language Model (served through Ollama) with a Model Context Protocol (MCP) server. Through EcoFlow, users can ask questions in natural language — for example, the current nitrogen level at a given station — and the assistant retrieves the real sensor data through read-only MCP tools and returns an understandable, contextualized answer.

## Development Methodology

The project will be developed using Scrum, an agile framework based on iterative and incremental delivery. Work is organized as a Product Backlog of User Stories — each traceable to a functional requirement — prioritized and estimated in Story Points using a Fibonacci-like scale (1, 2, 3, 5, 8...). The backlog is delivered across fixed-length Sprints; every Sprint ends with a Sprint Review (demo to stakeholders) and a Sprint Retrospective (process improvement).

### Product Backlog

| ID    | User Story                                                                                                                                                 | Points |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| US-01 | As a user, I want to log in with my username and password (RF-01), so that I can access the features assigned to my role.                                  | 3      |
| US-02 | As an administrator, I want to create, view, edit and deactivate user accounts (RF-02), so that I can manage who has access to the platform.               | 5      |
| US-03 | As an administrator, I want to assign a role to each user (RF-03), so that access to views and actions is controlled.                                      | 3      |
| US-04 | As an administrator, I want to register and manage geographic areas (RF-04), so that I can group monitoring stations by zone.                              | 3      |
| US-05 | As an administrator, I want to register and manage IoT stations identified by MAC address (RF-05), so that each station can authenticate and send data.    | 5      |
| US-06 | As an administrator, I want to register sensors with classification thresholds (RF-06), so that readings are automatically flagged as low, normal or high. | 5      |
| US-07 | As an administrator, I want to install or remove a sensor from a station (RF-07), so that only active sensors record samples.                              | 3      |
| US-08 | As an IoT node, I want to authenticate with my MAC address and submit readings continuously (RF-08), so that my data is persisted with a timestamp.        | 5      |
| US-09 | As a user, I want to see the latest reading per station in real time (RF-09), so that I can monitor current conditions.                                    | 5      |
| US-10 | As a user, I want to consult historical statistics through tables, cards and charts (RF-10), so that I can analyze trends over time.                       | 8      |

| ID    | User Story                                                                                                                                   | Points |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------|--------|
| US-11 | As a user, I want to generate a PDF report of the monitoring data (RF-11), so that I can share or archive it.                                | 5      |
| US-12 | As a user, I want to chat with EcoFlow in natural language (RF-12), so that I can understand environmental data without technical knowledge. | 8      |
| US-13 | As the system, I want EcoFlow to query data only through read-only MCP tools (RF-13), so that the LLM never accesses the database directly.  | 8      |

*Total backlog: 13 user stories, 66 story points.*

## Functional and non-functional requirements

### Functional

- RF-01 Login users (JWT)
- RF-02 CRUD users
- RF-03 CRUD user roles (Administrator, Normal)
- RF-04 CRUD geographic areas
- RF-05 CRUD IoT stations
- RF-06 CRUD sensors with classification thresholds
- RF-07 Install / remove sensors on a station
- RF-08 Receive and store samples from IoT nodes
- RF-09 View latest sample per station in real time
- RF-10 View historical statistics (tables, cards, charts)
- RF-11 Generate PDF monitoring reports
- RF-12 Natural-language chat with EcoFlow
- RF-13 Query system data through MCP tools

### Non-functional

- RNF-01 Security (JWT authentication on every request)
- RNF-02 Role-based access control
- RNF-03 Data integrity
- RNF-04 Performance
- RNF-05 Usability
- RNF-06 AI safety (read-only MCP access)

## Requirements analysis

### Requirements user administrator

**RF-01:** The system allows the administrator (and any registered user) to log in with a username and password. The API validates the credentials against the database and, if they are correct, issues a JWT that identifies the user's role for the rest of the session. If the credentials are invalid, the system informs the user without revealing which field was incorrect.

**RF-02:** The system must allow the administrator to register, consult, modify and deactivate user accounts.

**RF-03:** The system must allow the administrator to assign a role to each user — Administrator or Normal — controlling which views and actions that user can access.

**RF-04:** The system must allow the administrator to register, consult, modify and change the status of geographic areas, in order to group monitoring stations by zone.

**RF-05:** The system must allow the administrator to register, consult, modify and change the status of IoT stations. Each station is uniquely identified by its MAC address, which also serves as its authentication credential when sending samples.

**RF-06:** The system must allow the administrator to register, consult, modify and change the status of sensors, including the three classification thresholds (low, medium, high) used to automatically flag readings.

**RF-07:** The system must allow the administrator to install or remove a sensor from a station (an "armado"), which is the entity that enables that station to record samples for that sensor.

### Requirements — IoT node (system actor)

**RF-08:** The system must allow IoT nodes to authenticate with their MAC address and submit sensor readings continuously. Each submitted sample must be persisted with its timestamp for later consultation.

### Requirements user (public / normal user)

**RF-09:** The system must allow any authorized user to see, in real time, the most recent reading of every sensor installed at a station.

**RF-10:** The system must allow users to consult historical statistics of the stored samples through tables, summary cards and dynamic charts.

**RF-11:** The system must allow users to generate a PDF report of the monitoring data, including institutional headers and embedded charts.

**RF-12:** The system must offer a natural-language chat module — EcoFlow — where users can ask questions about the environmental and soil data in plain language.

**RF-13:** The system must resolve EcoFlow's queries exclusively through read-only MCP tools (`get_sensores`, `get_estaciones`, `get_roles`, `get_muestras_actuales`), so that the underlying language model never accesses the database directly.

## Non-requirements analysis

In general, the system must guarantee a high level of security for the information exchanged between the IoT nodes, the API and the web application, since all communication is authenticated with JWT tokens and credentials are stored using bcrypt hashing. Access to administrative views must be strictly controlled by the user's role. The platform must remain responsive as the number of stations and stored samples grows, registering and querying samples in under five seconds and answering EcoFlow queries in under seven seconds. Finally, the conversational assistant must never access the database directly — every data request must pass through read-only MCP tools — so that the integrity and trustworthiness of the monitoring data is preserved.

## Architecture of the system

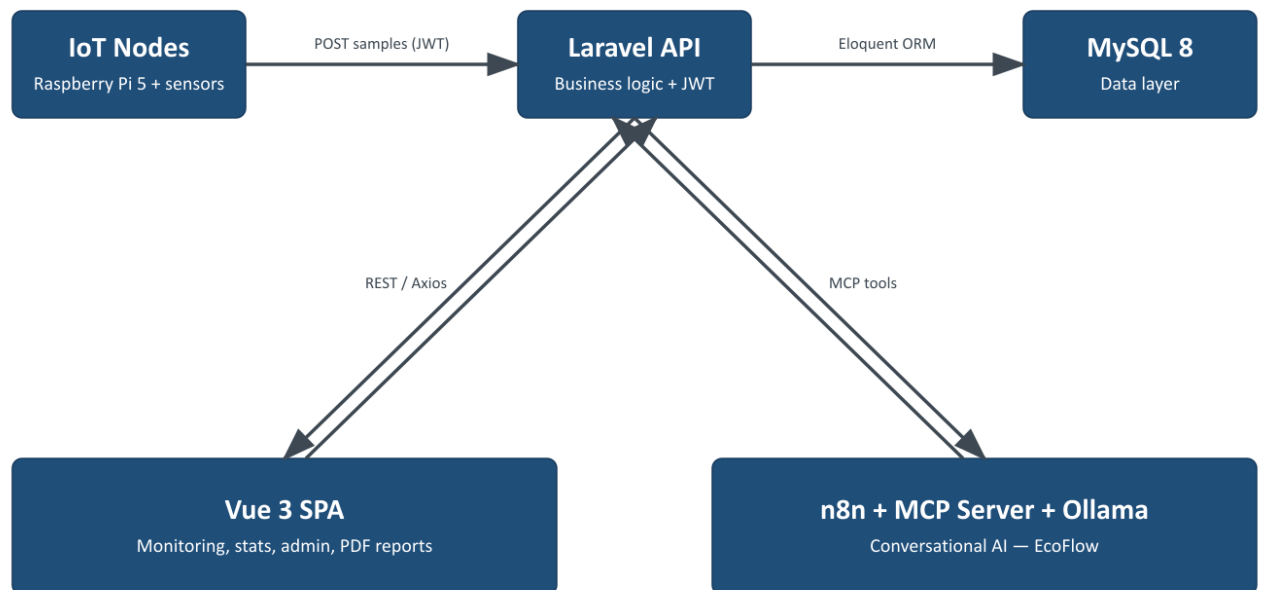


Fig. 1.1. SMMMyE five-layer system architecture

Fig. 1.1. System Architecture

## System modules

### 1. Authentication Module

- a. Functional requirements: RF-01
- b. Non functional: RNF-01, RNF-02

### 2. Users & Access Module

- a. Functional requirements: RF-02, RF-03
- b. Non functional: RNF-01, RNF-02, RNF-05

### 3. Catalog Module (Areas, Stations, Sensors)

- a. Functional requirements: RF-04, RF-05, RF-06, RF-07
- b. Non functional: RNF-03, RNF-04, RNF-05

### 4. Monitoring & Ingestion Module

- a. Functional requirements: RF-08, RF-09, RF-10, RF-11
- b. Non functional: RNF-03, RNF-04, RNF-05

### 5. AI Assistant Module (EcoFlow)

- a. Functional requirements: RF-12, RF-13
- b. Non functional: RNF-04, RNF-05, RNF-06

## Schedule of activities

Following the Scrum methodology described above, the 12-month roadmap is organized into 26 fixed two-week Sprints, grouped into seven stages.

| Stage                          | Sprints       | Weeks       | Description                                       |
|--------------------------------|---------------|-------------|---------------------------------------------------|
| 1. Baseline & Requirements     | Sprints 1–2   | Weeks 1–4   | Should be developed first                         |
| 2. SCM Plan & Config. ID       | Sprints 3–4   | Weeks 5–8   | Formalizes configuration items and the SCM plan   |
| 3. Architecture & UX/UI Design | Sprints 5–8   | Weeks 9–16  | System design and interface design system         |
| 4. Implementation              | Sprints 9–17  | Weeks 17–34 | Core development — see module breakdown below     |
| 5. Testing & Config. Audit     | Sprints 18–21 | Weeks 35–42 | Verification, validation and configuration audits |
| 6. Buffer / Contingency        | Sprints 22–25 | Weeks 43–50 | Explicit ≈15% schedule margin                     |
| 7. Final Delivery & Release    | Sprint 26     | Weeks 51–52 | Release and handover                              |

### Implementation stage — Sprint breakdown by module

| Module                        | Sprints | Weeks | Description                                                    |
|-------------------------------|---------|-------|----------------------------------------------------------------|
| Authentication Module         | 1       | 2     | Should be developed first                                      |
| Users & Access Module         | 1       | 2     | Manages accounts and role-based permissions                    |
| Catalog Module                | 2       | 4     | Areas, stations, sensors — core configuration identification   |
| Monitoring & Ingestion Module | 3       | 6     | Main module of the system; receives and visualizes sensor data |
| AI Assistant Module (EcoFlow) | 2       | 4     | Integrates n8n, MCP server and Ollama                          |

|       |   |    |  |
|-------|---|----|--|
| Total | 9 | 18 |  |
|-------|---|----|--|

## People involved

All developers are classified by seniority, and the Backend Developer works in a Fullstack capacity (Laravel API plus Vue support) to keep the team lean.

| Role                                  | Number | Salary MXN (monthly) | Total cost 12 months |
|---------------------------------------|--------|----------------------|----------------------|
| Project Manager (part-time)           | 1      | 10,000               | 120,000              |
| Backend Developer — Senior, Fullstack | 1      | 28,000               | 336,000              |
| Frontend Developer — Junior           | 1      | 14,000               | 168,000              |

| Role                                    | Number   | Salary MXN (monthly) | Total cost 12 months |
|-----------------------------------------|----------|----------------------|----------------------|
| UI / UX Designer — Mid-level, part-time | 1        | 12,000               | 144,000              |
| AI / Automation Engineer — Senior       | 1        | 30,000               | 360,000              |
| DevOps / QA — Junior, part-time         | 1        | 8,000                | 96,000               |
| <b>Team total</b>                       | <b>6</b> | <b>102,000</b>       | <b>1,224,000</b>     |

### Infrastructure costs

No item is treated as free: server hosting, GPU inference hardware and domain/SSL are all costed at market rates.

| Item                                    | Recurring cost | Cost (12 months) MXN |
|-----------------------------------------|----------------|----------------------|
| Server hosting (Docker + GPU-ready VPS) | 3,000 / mo     | 36,000               |
| GPU for local LLM inference (Ollama)    | — (one-time)   | 25,000               |
| Domain & SSL                            | — (annual)     | 500                  |

### Profit margin and total budget

Base project cost (team + infrastructure, 12 months): **\$1,285,500 MXN**

Profit margin (15%): **+ \$192,825 MXN**

**Total project budget: \$1,478,325 MXN**

A 15% profit margin is applied to the base cost, matching the ≈15% schedule contingency already reserved in the Buffer / Contingency stage (Sprints 22–25, Weeks 43–50) — the same margin is applied consistently to both money and time.